

Complete Jenkins Notes

1. Introduction to Jenkins

What is Jenkins?

Jenkins is an open-source automation server written in Java used for Continuous Integration and Continuous Deployment (CI/CD).

It helps automate:

- Build
- Testing
- Deployment
- Monitoring

What is CI/CD?



Continuous Integration (CI)

Developers frequently merge code into a central repository. Jenkins automatically:

- Pulls code
- Builds application
- Runs tests

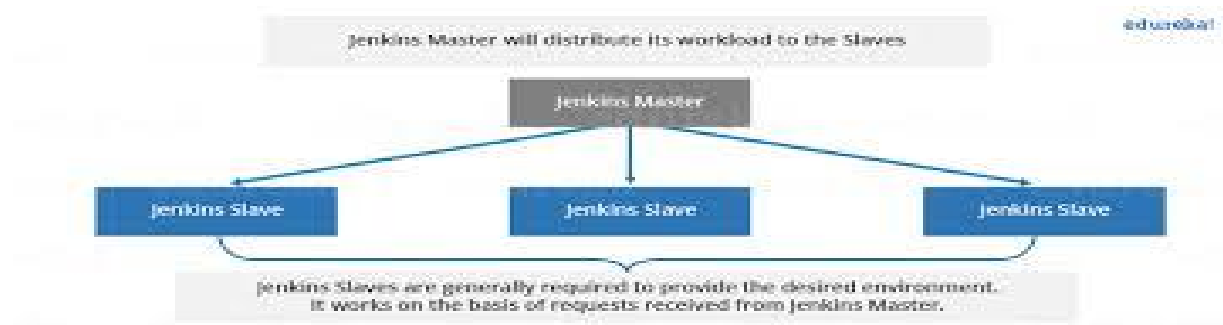
Continuous Deployment (CD)

After successful testing, the application is automatically deployed to production or remote servers.

CI/CD Flow

Code → Build → Test → Package → Deploy → Monitor

2. Jenkins Architecture



Components of Jenkins

Component	Purpose
Controller (Master)	Manages Jenkins
Agent (Slave/Node)	Executes jobs
Job	Automation task
Plugins	Extend Jenkins functionality
Pipeline	CI/CD workflow

Jenkins Controller Responsibilities

- Job scheduling
- Build management
- Plugin management
- Pipeline execution
- User management
- Security management

Jenkins Agent Responsibilities

- Executes builds
- Runs tests
- Deploys applications

Agents can run on:

- Linux
- Windows

- Docker
- Kubernetes

3. Jenkins Installation on Ubuntu

Step 1: Update Packages

```
sudo apt update
```

Step 2: Install Java

```
sudo apt install openjdk-11-jre -y
```

Verify Java:

```
java -version
```

Step 3: Install Jenkins

```
curl -fsSL https://pkg.jenkins.io/debian-stable/jenkins.io-2023.key | sudo tee \
/usr/share/keyrings/jenkins-keyring.asc > /dev/null
```

```
echo deb [signed-by=/usr/share/keyrings/jenkins-keyring.asc] \
https://pkg.jenkins.io/debian-stable binary/ | sudo tee \
/etc/apt/sources.list.d/jenkins.list > /dev/null
```

```
sudo apt update
sudo apt install jenkins -y
```

Step 4: Start Jenkins

```
sudo systemctl start jenkins
sudo systemctl enable jenkins
```

Step 5: Check Status

```
sudo systemctl status jenkins
```

Step 6: Access Jenkins

```
http://YOUR_IP:8080
```

Step 7: Get Initial Password

```
sudo cat /var/lib/jenkins/secrets/initialAdminPassword
```

4. Jenkins Dashboard Overview

Main sections:

- New Item
- Build History
- Manage Jenkins
- Credentials
- Nodes
- Plugins

5. Creating Jenkins Jobs

Freestyle Job

Freestyle project is a basic Jenkins job configured using GUI.

Used for:

- Build
- Testing
- Shell scripts
- Deployments

Steps to Create Freestyle Job

1. Dashboard → New Item
2. Enter Job Name
3. Select:
 - Freestyle Project
4. Click OK

Add Build Step

Build → Add Build Step → Execute Shell

```
echo "Build Started"  
date  
hostname
```

Run Build

Click:

Build Now

Console Output

Check:

Console Output

6. Types of Jenkins Jobs

Job Type	Purpose
Freestyle Project	Simple automation
Pipeline Project	CI/CD automation
Multibranch Pipeline	Builds multiple Git branches

7. Git & GitHub Integration

Install Git

```
sudo apt install git -y
```

Configure Git in Jenkins

Steps

1. Open Jenkins Job
2. Configure
3. Source Code Management
4. Select Git
5. Add Repository URL
6. Add Credentials if repository is private

Build Trigger

Poll SCM

Automatically checks repository changes.

Configure:

Build Triggers → Poll SCM

GitHub Webhook

Payload URL

`http://YOUR_JENKINS_URL/github-webhook/`

Steps in GitHub

1. Repository → Settings
2. Webhooks
3. Add Webhook
4. Paste Payload URL
5. Content Type:

`application/json`

8. Jenkins Pipeline

Pipeline automates CI/CD stages using code.

Types of Jenkins Pipeline

1. Declarative Pipeline

Structured and easy-to-read syntax.

Example

```
pipeline {  
  
    agent any  
  
    stages {  
  
        stage('Build') {  
            steps {  
                echo 'Build Stage'  
            }  
        }  
  
        stage('Test') {  
            steps {  
                echo 'Test Stage'  
            }  
        }  
  
        stage('Deploy') {  
            steps {  
                echo 'Deploy Stage'  
            }  
        }  
    }  
}
```

2. Scripted Pipeline

Advanced pipeline using Groovy scripting.

Example

```
node {  
  
    stage('Build') {  
        echo 'Build Stage'  
    }  
  
    stage('Test') {  
        echo 'Test Stage'  
    }  
  
    stage('Deploy') {  
        echo 'Deploy Stage'  
    }  
}
```

Core Pipeline Stages

Stage	Purpose
Build	Compile application
Test	Execute tests
Packag e	Create artifacts
Deploy	Deploy application

9. Jenkinsfile

Jenkinsfile stores pipeline code inside repository.

Example Structure

```
project/  
├── src/  
├── pom.xml  
└── Jenkinsfile
```

Benefits:

- Version control
- Reusable

- Easy collaboration

10. Maven with Jenkins

What is Maven?

Apache Maven is a build automation and dependency management tool mainly used for Java projects.

Install Maven

```
sudo apt install maven -y
```

Maven Commands

Build Application

```
mvn clean package
```

Run Tests

```
mvn test
```

11. Environment Variables

Common Jenkins environment variables:

Variable	Purpose
BUILD_NUMBER	Current build number
WORKSPACE	Current workspace path
JOB_NAME	Job name

Example

```
echo "${BUILD_NUMBER}"
```

12. Jenkins Parameters

Parameters allow user input during build execution.

Enable Parameters

1. Open Job
2. Configure
3. Select:

This project is parameterized

Types of Parameters

Type	Example
String	Version
Choice	Environment
Boolean	Deploy Yes/No

Example

```
parameters {  
    string(name: 'VERSION', defaultValue: '1.0')  
}
```

13. Scheduled Builds

Jenkins can run jobs automatically using CRON syntax.

Configure

1. Open Job
2. Configure
3. Build Triggers
4. Select:

Build periodically

Example

Every minute:

* * * * *

14. Email Notification Configuration

Install Plugin

Install:

Email Extension Plugin

Configure SMTP

Steps

1. Manage Jenkins
2. Configure System
3. E-mail Notification

Gmail SMTP Configuration

Setting	Value
SMTP Server	smtp.gmail.com
Port	587
Use SSL	Enabled
Authentication	Gmail App Password

15. Working on Remote Server Using Jenkins

Plugin Required

Install:

Publish Over SSH Plugin

Configure SSH Server

Steps

1. Manage Jenkins
2. Configure System
3. Publish over SSH

4. Add:

- Hostname
- Username
- SSH Key

Build Step

Choose:

Send files or execute commands over SSH

Example Command

```
systemctl restart nginx
```

16. Jenkins + Ansible

What is Ansible?

Ansible is an automation tool used for server configuration and deployment.

Install Plugin

Install:

Ansible Plugin

Example Playbook

```
- hosts: web  
  become: yes
```

```
tasks:
```

```
- name: Install Apache  
  apt:  
    name: apache2  
    state: present
```

Configure in Jenkins

Build Step:

Invoke Ansible Playbook

17. Role-Based Access Control (RBAC)

Plugin Required

Install:

Role-based Authorization Strategy

Configuration Steps

1. Manage Jenkins
2. Security
3. Authorization
4. Select:

Role-Based Strategy

Create Roles

Examples:

- Admin
- Developer
- Read-only

18. Jenkins Agent Configuration

Configure Agent

Steps

1. Manage Jenkins
2. Nodes
3. New Node

Configure Fields

Setting	Example
Node Type	Permanent Agent

Remote Root Directory C:\Users\dotnet\Desktop\slave

Labels windows

Download agent.jar

```
curl.exe -sO http://JENKINS_IP:8080/jnlpJars/agent.jar
```

Create Secret File

```
echo SECRET > secret-file
```

Connect Windows Agent

```
java -jar agent.jar -url http://JENKINS_IP:8080/ \
-secret @secret-file \
-name windows \
-webSocket \
-workDir "C:\Users\dotnet\Desktop\slave"
```

Common Issues During Agent Setup

Git Not Found Error

Cause

Git not installed or PATH not configured.

Fix

- Install Git
- Add Git to PATH
- Restart Jenkins agent

Access Denied Error

Cause

Jenkins agent not running as Administrator.

Fix

Run CMD as Administrator.

19. Jenkins + .NET Deployment

Pipeline Stages

Stage	Purpose
Restore	Download dependencies
Build	Compile application
Publish	Generate deployment files
Deploy	Deploy to IIS

Git Clone

git clone <https://github.com/niraj-nimap/dotnet-demo-project.git>

Restore Packages

dotnet restore

Build Application

dotnet build --configuration Release

Publish Application

dotnet publish -c Release -o publish

IIS Deployment

Deployment Path

C:\inetpub\wwwroot\MyDotnetApp

Create Application Pool

Setting	Value
Name	MyDotnetPool
CLR Version	No Managed Code

Create Website

Setting	Value
Site Name	MyDotnetApp
Physical Path	C:\inetpub\wwwroot\MyDotnetApp
Port	8081

20. Troubleshooting

HTTP Error 500.19

Cause

- Missing ASP.NET Core Hosting Bundle
- Invalid IIS configuration

Fix

- Install Hosting Bundle
- Restart IIS

Missing Runtime Error

Cause

Required .NET runtime missing.

Check Runtime

```
dotnet --list-runtimes
```

Restart IIS

iisreset

21. Important Learnings

- Jenkins architecture
- Freestyle jobs
- Git integration
- Jenkins pipelines
- GitHub webhook
- Jenkins agents
- Windows agent setup
- Maven integration
- .NET deployment
- IIS deployment
- Email notifications
- Ansible integration
- RBAC
- Troubleshooting Jenkins and IIS errors

Jenkins Shared Libraries

In Jenkins, **Shared Libraries** let you reuse common pipeline code across multiple projects instead of copying the same Jenkinsfile logic everywhere.

They are heavily used in DevOps teams to standardize:

- Build steps
- Docker image creation
- Kubernetes deployment
- Notifications
- Security scans
- Terraform automation
- CI/CD workflows

Why Shared Libraries Are Important

Without shared libraries:

```
pipeline {
```

```

stages {
  stage('Build') {
    steps {
      sh 'mvn clean package'
    }
  }
}

```

You repeat this in every repo.

With shared libraries:

```

@Library('devops-lib') _
buildJavaApp()

```

Much cleaner and reusable.

Types of Jenkins Shared Libraries

Type	Description
Global Library	Available to all Jenkins pipelines
Folder Library	Available only inside a folder
Automatic Library	Loaded dynamically

Shared Library Structure

A standard Jenkins shared library looks like this:

```

(devops-lib)
|
|— vars/
|   |— buildJavaApp.groovy
|   |— deployDocker.groovy
|   |— notifySlack.groovy
|
|— src/
|   |— com/devops/utils/
|       |— DockerUtils.groovy
|
|— resources/
|   |— templates/

```

```
|   └─ nginx.conf
|   └─ README.md
```

Important Folders

1. vars/

Contains reusable pipeline functions.

Example:

```
// vars/buildJavaApp.groovy
```

```
def call() {
    sh 'mvn clean package'
}
```

Usage:

```
@Library('devops-lib') _
```

```
pipeline {
    agent any

    stages {
        stage('Build') {
            steps {
                buildJavaApp()
            }
        }
    }
}
```

2. src/

Used for advanced Groovy classes.

Example:

```
// src/com/devops/utis/DockerUtils.groovy
```

```

package com.devops.utils

class DockerUtils implements Serializable {

    def steps

    DockerUtils(steps) {
        this.steps = steps
    }

    def buildImage(String image) {
        steps.sh "docker build -t ${image} ."
    }
}

```

Usage:

```

@Library('devops-lib') _

import com.devops.utils.DockerUtils

def dockerUtils = new DockerUtils(this)

dockerUtils.buildImage("myapp:v1")

```

3. resources/

Stores static files:

- YAML
- JSON
- Shell scripts
- Config templates

Example:

```

def content = libraryResource 'templates/nginx.conf'
writeFile file: 'nginx.conf', text: content

```

How to Configure Shared Library in Jenkins

Step 1 — Create Git Repository

Example repo:

<https://github.com/company/devops-lib.git>

Step 2 — Configure in Jenkins

Go to:

Manage Jenkins

→ System

→ Global Trusted Pipeline Libraries

Add:

Field	Value
Name	devops-lib
Default Version	main
Retrieval Method	Modern SCM
SCM	Git

Using Shared Library

Method 1 — Load Entire Library

```
@Library('devops-lib') _
```

Method 2 — Specific Version

```
@Library('devops-lib@dev') _
```

Real DevOps Examples

Example 1 — Docker Build Function

```
// vars/dockerBuild.groovy
```

```
def call(String imageName) {  
    sh "docker build -t ${imageName} ."  
}
```

Usage:

```
dockerBuild("payment-service:v1")
```

Example 2 — Kubernetes Deployment

```
// vars/k8sDeploy.groovy
```

```
def call(String manifest) {  
    sh "kubectl apply -f ${manifest}"  
}
```

Usage:

```
k8sDeploy("deployment.yaml")
```

Example 3 — Terraform Apply

```
// vars/tfApply.groovy
```

```
def call() {  
    sh ""  
        terraform init  
        terraform apply -auto-approve  
    ""  
}
```

Complete Real-World CI/CD Flow

```
@Library('devops-lib') _
```

```
pipeline {  
    agent any  
  
    stages {  
  
        stage('Build') {  
            steps {  
                buildJavaApp()  
            }  
        }  
  
        stage('Docker Build') {  
            steps {  
                dockerBuild("myapp:v1")  
            }  
        }  
    }  
}
```

```

    stage('Push Image') {
        steps {
            dockerPush("myapp:v1")
        }
    }

    stage('Deploy') {
        steps {
            k8sDeploy("k8s/deployment.yaml")
        }
    }
}

```

Advanced Concepts

Dynamic Library Loading

library identifier: 'devops-lib@main'

Access Environment Variables

env.BUILD_NUMBER
env.JOB_NAME

Use Credentials

```

withCredentials([
    usernamePassword(
        credentialsId: 'dockerhub',
        usernameVariable: 'USER',
        passwordVariable: 'PASS'
    )
]) {
    sh 'docker login'
}

```

Best Practices

Keep vars/ Small

Good:

```
buildApp()
deployApp()
```

Bad:

Huge 1000-line functions.

Use src/ for Complex Logic

Business logic belongs in classes.

Version Your Library

Use branches/tags:

```
@Library('devops-lib@v2') _
```

Write Documentation

Every function should have:

- Purpose
- Parameters
- Example usage

Common Interview Questions

What are Jenkins Shared Libraries?

Reusable Groovy code shared across Jenkins pipelines.

Difference Between vars and src?

vars	src
Simple pipeline functions	Complex Groovy classes
Auto-exposed	Requires import
Easier	More structured

What is `call()` in Shared Libraries?

Special method enabling direct function invocation.

Example:

```
myFunction()
```

instead of:

```
myFunction.call()
```

Real Enterprise Usage

Companies use shared libraries for:

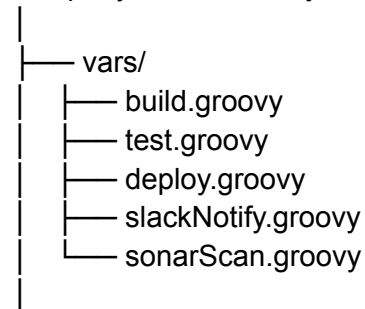
- Standard CI/CD
- Security scanning
- Docker workflows
- Kubernetes deployments
- Terraform automation
- Slack notifications
- SonarQube integration
- Multi-cloud deployment

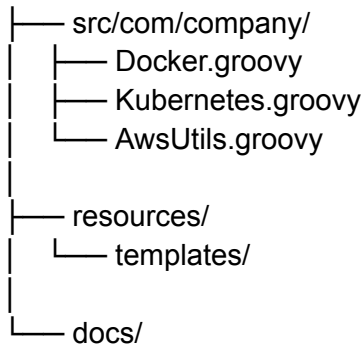
Important Jenkins Plugins

- Pipeline
- Git
- Docker
- Kubernetes
- Terraform
- SonarQube

Typical Enterprise Repository Structure

company-shared-library/





Jenkins Credentials Notes

In Jenkins, **Credentials** are used to securely store secrets required by pipelines and jobs.

Examples:

- GitHub username/password
- SSH private keys
- Docker Hub login
- AWS keys
- Kubernetes tokens
- API tokens
- Certificates

Instead of writing secrets directly inside Jenkinsfiles, Jenkins stores them securely in the credential store.

Why Jenkins Credentials Are Important

Without credentials:

```
sh 'docker login -u admin -p mypassword'
```

This is insecure because passwords are visible.

With Jenkins credentials:

```
withCredentials([
    usernamePassword(
        credentialsId: 'dockerhub',
```

```
        usernameVariable: 'USER',
        passwordVariable: 'PASS'
    )
  }) {
    sh 'docker login -u $USER -p $PASS'
  }
}
```

Much safer.

Where Credentials Are Managed

Go to:

Manage Jenkins
→ Credentials

You can store credentials:

Scope	Description
Global	Available to all jobs
System	Used internally by Jenkins
Folder	Available inside a folder
User	Personal credentials

Types of Jenkins Credentials

Credential Type	Used For
Username with Password	GitHub, Docker Hub
Secret Text	API tokens
Secret File	kubeconfig, JSON keys
SSH Username with Private Key	Git SSH access
Certificate	SSL/TLS authentication
AWS Credentials	AWS access

1. Username with Password

Used for:

- Docker Hub
- Nexus
- Artifactory
- GitHub HTTPS authentication

Example: Docker Hub Login

Add Credential

Kind: Username with password

ID: dockerhub

Username: mydockeruser

Password: mypassword

Pipeline Usage

```
withCredentials([
  usernamePassword(
    credentialsId: 'dockerhub',
    usernameVariable: 'USER',
    passwordVariable: 'PASS'
  )
]) {

  sh 'docker login -u $USER -p $PASS'
}
```

2. Secret Text

Stores:

- API tokens
- GitHub PAT
- Slack webhook
- JWT token

Example: GitHub Token

Add Credential

Kind: Secret text

ID: github-token

Secret: ghp_XXXXXX

Usage

```
withCredentials([
  string(
    credentialsId: 'github-token',
    variable: 'TOKEN'
  )
]) {

  sh 'curl -H "Authorization: token $TOKEN" https://api.github.com/user'
}
```

3. Secret File

Stores files like:

- kubeconfig
- AWS JSON key
- GCP service account
- SSL cert files

Example: Kubernetes kubeconfig

Add Credential

Kind: Secret file

ID: kubeconfig

Upload kubeconfig file.

Usage

```
withCredentials([
  file(
    credentialsId: 'kubeconfig',
    variable: 'KUBECONFIG_FILE'
  )
]) {
```

```
    sh 'kubectl --kubeconfig=$KUBECONFIG_FILE get pods'
}
```

4. SSH Username with Private Key

Used for:

- Git SSH clone
- Remote server access
- Ansible
- SCP/SSH automation

Example: Git Clone Using SSH

Add Credential

Kind: SSH Username with private key

ID: git-ssh

Username: git

Private Key: ~/.ssh/id_rsa

Usage

```
git credentialsId: 'git-ssh',
url: 'git@github.com:company/project.git'
```

Example: SSH into Server

```
withCredentials([
    sshUserPrivateKey(
        credentialsId: 'server-ssh',
        keyFileVariable: 'SSH_KEY',
        usernameVariable: 'SSH_USER'
    )
]) {

    sh """
        ssh -i $SSH_KEY $SSH_USER@10.0.0.5 "uptime"
    """
}
```

5. Certificate Credentials

Used for:

- SSL authentication
- HTTPS mutual TLS
- Java keystore authentication

Example Usage

```
withCredentials([
  certificate(
    credentialsId: 'ssl-cert',
    keystoreVariable: 'KEYSTORE',
    passwordVariable: 'PASSWORD'
  )
]) {

  sh 'java -Djavax.net.ssl.keyStore=$KEYSTORE app.jar'
}
```

6. AWS Credentials

Used for:

- Terraform
- AWS CLI
- EKS
- S3
- EC2 deployment

Add AWS Credential

Install:

AWS Credentials Plugin

Usage

```
withCredentials([
  [$class: 'AmazonWebServicesCredentialsBinding',
    credentialsId: 'aws-creds']
]) {

  sh 'aws s3 ls'
}
```

7. Kubernetes Token Credentials

Used for:

- Kubernetes API access
- CI/CD deployment

Example

```
withCredentials([
  string(
    credentialsId: 'k8s-token',
    variable: 'TOKEN'
  )
]) {

  sh '''
    curl -H "Authorization: Bearer $TOKEN" \
    https://k8s-api-server
  '''
}
```

Credential Usage in Declarative Pipeline

Full Example

```
pipeline {
  agent any

  stages {
    stage('Docker Login') {
      steps {
        withCredentials([
          usernamePassword(
            credentialsId: 'dockerhub',
            usernameVariable: 'USER',
            passwordVariable: 'PASS'
          )
        ])
      }
    }
  }
}
```

```
}) {  
    sh 'docker login -u $USER -p $PASS'  
}  
}  
}
```

Credential Usage in Scripted Pipeline

```
node {
  withCredentials([
    string(
      credentialsId: 'api-token',
      variable: 'TOKEN'
    )
  ]) {
    sh 'echo $TOKEN'
  }
}
```

Environment Variable Method

Jenkins can expose credentials as environment variables.

Example:

```
environment {
  AWS_ACCESS_KEY_ID = credentials('aws-access-key')
}
```

Credentials Binding Plugin

Important plugin:

Credentials Binding Plugin

Used to inject secrets into pipelines securely.

Real DevOps Examples

Example 1 — Docker Push

```
withCredentials([
  usernamePassword(
    credentialsId: 'dockerhub',
    usernameVariable: 'USER',
    passwordVariable: 'PASS'
  )
]) {

  sh """
    docker login -u $USER -p $PASS
    docker push myapp:v1
  """
}
```

Example 2 — Terraform with AWS

```
withCredentials([
  [$class: 'AmazonWebServicesCredentialsBinding',
   credentialsId: 'aws-creds']
]) {

  sh """
    terraform init
    terraform apply -auto-approve
  """
}
```

Example 3 — Kubernetes Deployment

```
withCredentials([
  file(
    credentialsId: 'kubeconfig',
    variable: 'KUBECONFIG'
  )
]) {

  sh 'kubectl get nodes'
}
```

Example 4 — Git Clone

```
git branch: 'main',  
credentialsId: 'git-ssh',  
url: 'git@github.com:company/app.git'
```

Credential Security Best Practices

Never Hardcode Secrets

Bad:

```
sh 'docker login -u admin -p password'
```

Good:

Use Jenkins credentials.

Use Least Privilege

Give minimum permissions needed.

Example:

- Read-only AWS access if deployment is not needed.

Rotate Secrets Regularly

Change:

- Tokens
- Passwords
- SSH keys

Use Folder-Level Credentials

Avoid exposing production secrets globally.

Avoid Printing Secrets

Bad:

```
echo $PASSWORD
```

Important Jenkins Plugins

Plugin	Purpose
Credentials Plugin	Core credential management
Credentials Binding Plugin	Inject credentials into pipelines
SSH Agent Plugin	SSH authentication
AWS Credentials Plugin	AWS integration
Kubernetes Plugin	K8s integration

Common Interview Questions

What are Jenkins Credentials?

Secure storage for secrets used by Jenkins jobs and pipelines.

Why use **withCredentials**?

Temporarily injects secrets into environment variables securely.

Difference Between Secret Text and Secret File?

Secret Text	Secret File
Tokens/passwords	Files/certificates
Small secrets	Large file-based secrets

Difference Between Username/Password and SSH Key?

Username/Password	SSH Key
HTTPS authentication	SSH authentication
Simpler	More secure

Enterprise Credential Flow

Typical CI/CD flow:

GitHub → Jenkins → Docker Hub → Kubernetes → AWS

Credentials used:

Service	Credential Type
GitHub	SSH Key
Docker Hub	Username/Password
AWS	AWS Credentials
Kubernetes	kubeconfig
Slack	Secret Text